

6-ma'ruza: Operatorlarni qayta yuklash.

Reja:

1. Operatorni qayta yuklash asoslari
2. Binar va unar peratorlarni qayta yuklash
3. Unar operatorlarini qayta yuklash

C# tili operatorni yaratilgan sinfga nisbatan aniqlash imkonini beradi. Bu jarayon operatorning *qayta yuklanishi* deb ataladi. Qayta yuklash sinfdagi operator doirasini kengaytiradi. Shu bilan birga, operatorning bajarilishi to'liq nazorat qilinadi va muayyan sinfga qarab farq qilishi mumkin. Masalan, + operatori obyektlarni bog'lash uchun ishlatilsa, boshqa sinfdan esa uning vazifasi butunlay boshqacha bo'lishi mumkin.

Operator qayta yuklanganda, uning dastlabki holatidan hech biri yo'qolmaydi. U shunchaki ma'lum bir obyektga boshqa yangi operatsiyani bajaradi. Shuning uchun + operatorini qayta yuklash, masalan, obyektlarni qayta ishlash uchun uning butun sonlarga nisbatan tayinlanishini o'zgartirmaydi, ya'ni, ularni faqat qo'shish uchun xizmat qiladi.

Operatorni qayta yuklashning asosiy afzalligi shundaki, u yangi turdagi sinfni dasturlash muhitiga muammosiz integratsiya qilish imkonini beradi. Bu C# kabi ob'ektga yo'naltirilgan dasturlash tili samaradorligining muhim qismidir.

Operatorni qayta yuklash asoslari

Operatorning qayta yuklanishi metodni qayta yuklash bilan chambarchas bog'liq. Operatorni qayta yuklash uchun operator metodini aniqlaydigan **operator** kalit so'zi ishlatiladi, bu esa o'z navbatida operatorning o'z sinfiga nisbatan harakatlarini aniqlaydi.

Operator metodlarining ikkita shakli mavjud: biri unar operatorlar uchun, ikkinchisi binar operatorlari uchun. Quyida ushbu metodlarning har bir o'zgarishi uchun umumiy shakl keltirilgan.

Unar operatorni qayta yuklashning umumiy shakli:

```
public static <qaytaruvchi_tur> operator <op>(<parametr_turi> operand)
{
    // operatsiya
}
```

Binar operatorni qayta yuklashning umumiy shakli:

```
public static <qaytaruvchi_tur> operator <op>(<parametr_turi1> operand1,
    <parametr_turi2> operand2) {
    // operatsiya
}
```

Bu yerda op o'rniga qayta yuklanuvchi operator yoziladi, masalan, + yoki / va <qaytaruvchi_tur> belgilangan operatsiya bilan qaytarilgan qiymatning o'ziga xos turini bildiradi. Ushbu qiymat har qanday turda bo'lishi mumkin, lekin ko'pincha u operator tomonidan qayta yuklangan sinf bilan bir xil bo'ladi. Ushbu jarayon qayta yuklangan operatorlarni ifodalarda ishlatishni osonlashtiradi. Unar operatorlari uchun operand uzatiladigan operandni, binar operatorlar uchun esa operand1 va operand2 bilan belgilanadi. Yana shunga e'tibor berish kerakki, operator metodlari public va static modifikatorlari orqali e'lon qilinishi kerak.

Unar operatorlarining operand turi operator qayta yuklangan sinf nomi bilan bir xil bo'lishi kerak. Binar operatorlarda kamida bitta operand shu sinf bilan bir xil bo'lishi kerak. Shunday qilib, C # hali o'rnatilmagan ob'ektlar uchun biron bir operatorni qayta yuklashga yo'l qo'ymaydi. Masalan, "+" operatorining tayinlanishi int yoki string tipidagi elementlar uchun bekor qilinishi mumkin emas. **Eslatma:** Qayta yuklash operator parametrlarida ref yoki out modifikatoridan foydalanib bo'lmaydi.

Binar operatorni qayta yuklash

Operatorning qayta yuklanishi qanday ishlashini namoyish qilish uchun, ikkita + va - operatorlarini qayta yuklaydigan oddiy misoldan boshlaylik. Quyidagi dastur ThreeD sinfdagi ob'ektning koordinatalarini o'z ichiga olgan ThreeD sinfini yaratadi. Qayta yuklangan +

operator bitta uch o'lchovli ob'ektning boshqa koordinatalarini qo'shadi. Va qayta yuklangan operator bitta ob'ektning koordinatalarini boshqa ob'ektning koordinatalaridan ajratib oladi.

```
using System;
//3D koordinatalarini saqlash uchun sinf yaratish
class ThreeD {
    int x, y, z; // uch o'lchovli koordinatalar
    public ThreeD() { x = y = z = 0; }
    public ThreeD(int i, int j, int k) { x = i; y = j; z = k; }
    // ikkita operatorni qayta yuklaydigan + binar operator
    public static ThreeD operator +(ThreeD op1, ThreeD op2)
    {
        ThreeD result = new ThreeD();
        /* Ikki nuqta koordinatalarini qo'shing va natijani qaytaring. */
        result.x = op1.x + op2.x;
        result.y = op1.y + op2.y;
        result.z = op1.z + op2.z;
        return result;
    }
    // ikkita operatorni qayta yuklaydigan - binar operator
    public static ThreeD operator -(ThreeD op1, ThreeD op2)
    {
        ThreeD result = new ThreeD();
        /* Operandlarning tartibiga e'tibor bering:
        op1 - chap operand va op2 - o'ng operand. */
        result.x = op1.x - op2.x;
        result.y = op1.y - op2.y;
        result.z = op1.z - op2.z;
        return result;
    }
    // X, Y, Z koordinatalarni ekranga chiqarish.
    public void Show()
    {
        Console.WriteLine(x + ", " + y + ", " + z);
    }
}
class ThreeDDemo {
    static void Main() {
        ThreeD a = new ThreeD(11, 2, 3);
        ThreeD b = new ThreeD(12, 10, 10);
        ThreeD c;
        Console.WriteLine("a nuqtaning koordinatalari: ");
        a.Show();
        Console.WriteLine ();
        Console.WriteLine("b nuqtaning koordinatalari: ");
        b.Show();
        Console.WriteLine ();
        c = a + b; // a va b nuqtalarning koordinatalarini qo'shing
        Console.WriteLine("a+b nuqtaning koordinatalari: ");
        c.Show();
        Console.WriteLine();
        c = a + b + c; // a, b va c nuqtalarning koordinatalarini qo'shing
        Console.WriteLine("a+b+c nuqtaning koordinatalari: ");
        c.Show();
        Console.WriteLine();
    }
}
```

```

c = c - a; //c nuqtadan a nuqtaning koordinatalarini ayting
Console.Write("c-a nuqtaning koordinatalari: ");
c.Show();
Console.WriteLine() ;
c = c - b; // c nuqtadan b nuqtaning koordinatalarini ayting
Console.Write("c-b nuqtaning koordinatalari: ");
c.Show();
Console.WriteLine() ;
Console.ReadKey();
}
}

```

Natija quyidagicha:

```

a nuqtaning koordinatalari: 11, 2, 3
b nuqtaning koordinatalari: 12, 10, 10
c=a+b nuqtaning koordinatalari: 23, 12, 13
c=a+b+c nuqtaning koordinatalari: 46, 24, 26
c=c-a nuqtaning koordinatalari: 35, 22, 23
c=c-b nuqtaning koordinatalari: 23, 12, 13

```

Unar operatorlarini qayta yuklash

Unar operatorlari ham binar operatorlar kabi qayta yuklanadi. Asosiy farq, ularda faqat bitta operand mavjud. Misol sifatida, quyida ThreeD klassi uchun unar minus operatorini qayta yuklaydigan metod keltirilgan.

```

public static ThreeD operator -(ThreeD op)
{
    ThreeD result = new ThreeD();
    result.x = -op.x;
    result.y = -op.y;
    result.z = -op.z;
    return result;
}

```

Ushbu misol yangi ob'ektni yaratadi, uning maydonlarida qayta yuklangan unar operatorining operand va manfiy qiymatlari saqlanadi, shundan so'ng ushbu ob'ekt operator metodi bilan qaytariladi. E'tibor bering, operandning o'zi o'zgarmaydi. Bu shuni anglatadiki, bu holda, bir martalik minus operatorining odatiy maqsadi saqlanib qoladi. Masalan, ifoda natijasi.

a = -b

b operandning manfiy qiymati a ga qiymatlanadi, lekin b operandning o'zi o'zgarmaydi.

C # da ++ va -- operatorlarini qayta yuklash juda oddiy. Buning uchun inkrement yoki dekrement qiymatni qaytarish kifoya qiladi, ammo chaqiriluvchi o'zgartirmaydi. C # kompilyatori ushbu operatorlarning prefiks va postfiks shakllarini ajratib turadigan qolgan qismiga ishlov beradi. Misol tariqasida, quyida ThreeD klassi uchun ++ () operatori ko'rib chiqamiz.

```

// ++ unar operatorini qayta yuklash
public static ThreeD operator ++(ThreeD op)
{
    ThreeD result = new ThreeD();
    // inkrement natijasini qaytaring.
    result.x = op.x + 1;
    result.y = op.y + 1;
    result.z = op.z + 1;
    return result;
}

```

Quyidagi dastur kodi oldingi namunaviy dasturning kengaytirilgan versiyasi bo'lib, u ++ operatorining qayta yuklanishini ko'rsatadi.

// binar va unar operatorlarning qayta yuklanishiga misol

```

using System;
// Uch o'lchovli koordinatalarni saqlash uchun sinf
class ThreeD
{
    int x, y, z; // 3D koordinatalari
    public ThreeD() { x = y = z = 0; }
    public ThreeD(int i, int j, int k) { x = i; y = j; z = k; }
    // binar + operatorni qayta yuklang.
    public static ThreeD operator +(ThreeD op1, ThreeD op2)
    {
        ThreeD result = new ThreeD();
        /* Ikkala nuqtaning koordinatalarini qo'shing va natijani qaytaring*/
        result.x = op1.x + op2.x;
        result.y = op1.y + op2.y;
        result.z = op1.z + op2.z;
        return result;
    }
    // Binar - operatorni qayta yuklash
    public static ThreeD operator -(ThreeD op1, ThreeD op2)
    {
        ThreeD result = new ThreeD();
        /* Operandlarning tartibiga e'tibor bering:
        op1 - chap operand, op2 - o'ng operand*/
        result.x = op1.x - op2.x;
        result.y = op1.y - op2.y;
        result.z = op1.z - op2.z;
        return result;
    }
    // Unar - operatorni qayta yuklash
    public static ThreeD operator -(ThreeD op)
    {
        ThreeD result = new ThreeD();
        result.x = -op.x;
        result.y = -op.y;
        result.z = -op.z;
        return result;
    }
    // Unar ++ operatorni qayta yuklash
    public static ThreeD operator ++(ThreeD op)
    {
        ThreeD result = new ThreeD();
        // inkrement natijasini qaytaring
        result.x = op.x + 1;
        result.y = op.y + 1;
        result.z = op.z + 1;
        return result;
    }
    // X, Y, Z koordinatalarini chiqarish
    public void Show()
    {
        Console.WriteLine("(" + x + ", " + y + ", " + z + ")");
    }
}
class ThreeDDemo
{

```

```

static void Main()
{
    ThreeD a = new ThreeD(1, 2, 3);
    ThreeD b = new ThreeD(10, 10, 10);
    ThreeD c = new ThreeD();
    Console.Write("a nuqta koordinatalari: ");
    a.Show();
    Console.WriteLine();
    Console.Write("b nuqta koordinatalari: ");
    b.Show();
    Console.WriteLine();
    c = a + b; // a va b nuqtalarning koordinatalarini qo'shish
    Console.Write("a + b = ");
    c.Show();
    Console.WriteLine();
    c = a + b + c; // a, b va c nuqtalarning koordinatalarini qo'shing
    Console.Write("a + b + c = ");
    c.Show();
    Console.WriteLine();
    c = c - a; // a nuqta koordinatalarini ayirish
    Console.Write("c - a = ");
    c.Show();
    Console.WriteLine();
    c = c - b; // b nuqta koordinatalarini ayirish
    Console.Write("c - b = ");
    c.Show();
    Console.WriteLine();
    c = -a; // a nuqtaning manfiy koordinatalari bilan belgilash
    Console.Write("-a = ");
    c.Show();
    Console.WriteLine();
    c = a++; // a nuqta koordinatalaridan c nuqtaga belgilang,
    // va keyin a ni bittaga oshiring Console.WriteLine("c = a++");
    Console.Write("c nuqta koordinatalari teng bo'ladi: ");
    c.Show();
    Console.Write("a nuqtaning koordinatalari quyidagicha: ");
    a.Show();
    // a nuqtaning boshlang'ich koordinatalarini (1,2,3) o'rnating
    a = new ThreeD(1, 2, 3);
    Console.Write("\na nuqtaning boshlang'ich koordinatalarini chiqarish: ");
    a.Show();
    c = ++a; // a nuqta koordinatalarini bittaga oshiring,
    /*va keyin ularni c nuqtasiga belgilang Console.WriteLine("\nAgar c = ++a"); */
    Console.Write("u holda c nuqta koordinatalari quyidagiga teng bo'ladi ");
    c.Show();
    Console.Write("a nuqtaning koordinatalari quyidagicha ");
    a.Show();
    Console.ReadKey();
}
}

```

Ushbu dastur bajarilishi natijasida quyidagi natijani qaytaradi.

a nuqta koordinatalari: (1, 2, 3)

b nuqta koordinatalari: (10, 10, 10)

a + b = (11, 12, 13)

$a + b + c = (22, 24, 26)$

$c - a = (21, 22, 23)$

$c - b = (11, 12, 13)$

$-a = (-1, -2, -3)$

c nuqta koordinatalari teng bo'ladi: (1, 2, 3)

a nuqtaning koordinatalari quyidagicha: (2, 3, 4)

a nuqtaning boshlang'ich koordinatalarini chiqarish: (1, 2, 3)

u holda c nuqta koordinatalari quyidagiga teng bo'ladi (2, 3, 4)

a nuqtaning koordinatalari quyidagicha (2, 3, 4)

Arifmetik operatorlarni qayta yuklash. Har qanday sinf va operator uchun operator usulining o'zini ortiqcha yuklash ham mumkin. Bu, xususan, sinf turi va boshqa ma'lumotlar turlari, shu jumladan o'rnatilgan ma'lumotlar bo'yicha operatsiyalarga ruxsat berish uchun talab qilinadi. Keling, yana ThreeD sinfini ko'rib chiqaylik. Ushbu klassni misol sifatida ishlatib, biz bir ThreeD ob'ektning koordinatalarini boshqasining koordinatalariga qo'shish uchun + operatori qanday haddan tashqari yuklanganligini ko'rsatdik. Ammo bu ThreeD sinfi uchun qo'shimcha operatsiyani aniqlashning yagona usuli emas. Shunday qilib, ThreeD tipidagi ob'ektning har bir koordinatasiga butun son qiymatini qo'shish teng darajada foydali bo'ladi. Shunga o'xshash operatsiya koordinata o'qlarini o'tkazish uchun foydali bo'ladi. Lekin buni amalga oshirish uchun quyida ko'rsatilganidek, + operatorini yana ortiqcha yuklashingiz kerak.

// ThreeD tipidagi obyektini va int tipidagi butun son qiymatini qo'shish uchun + binar operatorini qayta yuklash

```
public static ThreeD operator +(ThreeD op1, int op2)
```

```
{
```

```
    ThreeD result = new ThreeD();
```

```
    result.x = op1.x + op2;
```

```
    result.y = op1.y + op2;
```

```
    result.z = op1.z + op2;
```

```
    return result;
```

```
}
```

Ko'rib turganingizdek, operator metodining ikkinchi parametri aynan int tipida, shuning uchun bu metod ThreeD tipidagi obyektning har bir maydoniga butun son qiymatini qo'shish imkonini beradi. Bunday operatsiya to'liq amal qiladi, chunki yuqorida aytib o'tilganidek, binar operator qayta yuklanganda, uning operandlaridan biri ushbu operator qayta yuklangan sinf bilan bir xil turda bo'lishi kerak. Ammo bu operatorning ikkinchi operandi har qanday boshqa turda bo'lishi mumkin.

Quyida + operatorining ikkita qayta yuklangan metodi bilan ThreeD sinfining varianti keltirilgan.

// + binar operator ikki marta qayta yuklash

/* ThreeD sinfi ob'ektlarini qo'shish uchun bir marta hamda ThreeD tipidagi obyektini va int tipidagi qiymatni qo'shish uchun yana bir marta qayta yuklash*/

```
using System;
```

```
// Uch o'lchovli koordinatalarni saqlash uchun sinf yaratish
```

```
class ThreeD
```

```
{
```

```
    int x, y, z; // Uch o'lchovli koordinatalar
```

```
    public ThreeD() { x = y = z = 0; }
```

```
    public ThreeD(int i, int j, int k) { x = i; y = j; z = k; }
```

```
    // ThreeD sinfi ob'ektlarini qo'shish uchun + ikkilik operatorini qayta yuklash
```

```
    public static ThreeD operator +(ThreeD op1, ThreeD op2)
```

```
{
```

```
    ThreeD result = new ThreeD();
```

```
    /* Ikki nuqtaning koordinatalarini qo'shish va natijani olish*/
```

```

result.x = op1.x + op2.x;
result.y = op1.y + op2.y;
result.z = op1.z + op2.z;
return result;
}
/* ThreeD tipidagi obyekttni va int tipidagi butun son qiymatini qo'shish uchun + binar
operatorini qayta yuklash*/
public static ThreeD operator +(ThreeD op1, int op2)
{
    ThreeD result = new ThreeD();
    result.x = op1.x + op2;
    result.y = op1.y + op2;
    result.z = op1.z + op2;
    return result;
}
// X, Y, Z koordinatalarini chiqarish.
public void Show()
{
    Console.WriteLine("(" + x + ", " + y + ", " + z + ")");
}
}
class ThreeDDemo
{
    static void Main()
    {
        ThreeD a = new ThreeD(1, 2, 3);
        ThreeD b = new ThreeD(10, 10, 10);
        ThreeD c = new ThreeD();
        Console.WriteLine("a nuqta koordinatalari: ");
        a.Show();
        Console.WriteLine("b nuqta koordinatalari: ");
        b.Show();
        c = a + b; // a va b nuqtalarning koordinatalarini qo'shish
        Console.WriteLine("a + b = ");
        c.Show();
        c = b + 10; // ThreeD tipidagi obyektga int tipidagi qiymatni qo'shish
        Console.WriteLine("b + 10 = ");
        c.Show();
        Console.ReadKey();
    }
}
Dastur bajarilishi natijasida quyidagi natija olinadi.
a nuqta koordinatalari: (1, 2, 3)
b nuqta koordinatalari: (10, 10, 10)
a + b = (11, 12, 13)
b + 10 = (20, 20, 20)

```

Yuqoridagi natija ma'lumki, + operatori ThreeD sinfidagi ikkita obyektga qo'llanilganda, ularning koordinatalari qo'shiladi hamda u ThreeD tipidagi obyektga va butun qiymatga qo'llanilganda, ushbu obyektning koordinatalari berilgan butun qiymatga oshiriladi.

Yuqorida ko'rsatilgan + operatorining qayta yuklanishi, ThreeD sinfining xususiyatlarini kengaytiradi, lekin u buni to'liq bajar olmaydi. Operator+(ThreeD, int) metodi quyidagi amallarni bajarish imkonini beradi.

```
ob1 = ob2 + 10;
```

Lekin, u quyidagi operatsiyalarni bajarishga imkon bermaydi.

```
ob1 = 10 + ob2;
```

Buning sababi shundaki, bu metodning ikkinchi butun argumenti binar operatorning + o'ng operandidir, lekin kodning yuqoridagi satrida butun son argumenti chap tomonda joylashgan. Bunday qo'shish operatsiyasiga ruxsat berish uchun siz yana + operatorini qayta yuklashingiz kerak. Bunda operator metodining birinchi parametri int tipidagi, ikkinchi parametri esa ThreeD tipidagi bo'lishi kerak. Shunday qilib, operator+() metodining bir variantida ThreeD tipidagi obyekt va butun son qiymatini qo'shish, ikkinchisida esa butun son qiymatini va ThreeD tipidagi obyekt qo'shish amalga oshiriladi. + operatorining qayta yuklanishi tufayli ushbu operatorning chap va o'ng tomonida o'rnatilgan ma'lumotlar turi paydo bo'lishiga ruxsat beriladi. Quyida ThreeD sinfining yana bir varianti keltirilgan, unda binar operator + yuqorida tavsiflangan tarzda qayta yuklangan.

```
using System;
```

```
// Uch o'lchovli koordinatalarni saqlash uchun sinf yaratish
```

```
class ThreeD
```

```
{
```

```
int x, y, z; // Uch o'lchovli koordinatalar
```

```
public ThreeD() { x = y = z = 0; }
```

```
public ThreeD(int i, int j, int k) { x = i; y = j; z = k; }
```

```
// ThreeD sinfi obyektlarini qo'shish uchun + binar operatorini qayta yuklash
```

```
public static ThreeD operator +(ThreeD op1, ThreeD op2)
```

```
{
```

```
ThreeD result = new ThreeD();
```

```
/* Ikki nuqtaning koordinatalarini qo'shish va natijani olish*/
```

```
result.x = op1.x + op2.x;
```

```
result.y = op1.y + op2.y;
```

```
result.z = op1.z + op2.z;
```

```
return result;
```

```
}
```

```
/* ThreeD tipidagi obyekt va int tipidagi son
```

```
qiymatiga qo'shish uchun + binar operatorini qayta yuklash*/
```

```
public static ThreeD operator +(ThreeD op1, int op2)
```

```
{
```

```
ThreeD result = new ThreeD();
```

```
result.x = op1.x + op2;
```

```
result.y = op1.y + op2;
```

```
result.z = op1.z + op2;
```

```
return result;
```

```
}
```

```
/* int tipidagi son qiymatini va ThreeD tipidagi obyektga
```

```
qo'shish uchun + binar operatorini qayta yuklash*/
```

```
public static ThreeD operator +(int op1, ThreeD op2)
```

```
{
```

```
ThreeD result = new ThreeD();
```

```

result.x = op2.x + op1;
result.y = op2.y + op1;
result.z = op2.z + op1;
return result;
}
// X, Y, Z koordinatalarini chiqarish.
public void Show()
{
    Console.WriteLine("(" + x + ", " + y + ", " + z + ")");
}
}
class ThreeDDemo {
    static void Main()
    {
        ThreeD a = new ThreeD(1, 2, 3);
        ThreeD b = new ThreeD(10, 10, 10);
        ThreeD c = new ThreeD();
        Console.WriteLine("a nuqta koordinatalari: ");
        a.Show();
        Console.WriteLine("b nuqta koordinatalari: ");
        b.Show();
        c = a + b; // a va b nuqtalarning koordinatalarini qo'shish
        Console.WriteLine("a + b = ");
        c.Show();
        c = b + 10; // ThreeD tipidagi obyektga int tipidagi qiymatni qo'shish
        Console.WriteLine("b + 10 = ");
        c.Show();
        c = 15 + b; // int tipidagi qiymatni va ThreeD tipidagi obyektini qo'shish
        Console.WriteLine("15 + b = ");
        c.Show();
        Console.ReadKey();
    }
}

```

Dastur bajarilishi natijasida quyidagi natija olinadi.

a nuqta koordinatalari: (1, 2, 3)

b nuqta koordinatalari: (10, 10, 10)

a + b = (11, 12, 13)

b + 10 = (20, 20, 20)

15 + b = (25, 25, 25)

Munosabat operatorining qayta yuklanishi. == va < kabi munosabat operatorlari ham juda oson yuklanishi mumkin. Odatda, qayta yuklangan munosabat operator mantiqiy true va false ni qaytaradi. Bu bunday operatorlarning odatiy qo'llanilishiga mos keladi va ularning qayta yuklangan tarmoqlarini shartli ifodalarda ishlatishga imkon beradi. Agar boshqa turdagi natija qaytarilsa, munosabat operatorlarning qo'llanilishi keskin cheklanadi.

Quyida < va > operatorlarini qayta yuklaydigan ThreeD sinfining yana bir versiyasi keltirilgan. Berilgan misolda ushbu operatorlar uch o'lchamli ob'ektlarni boshlang'ich nuqtadan masofasiga qarab solishtirish uchun ishlatiladi. Agar u boshlang'ich koordinataga nisbatan ikkinchi obyektдан uzoqda bo'lsa katta deb hisoblanadi, agar u boshlang'ich koordinataga nisbatan ikkinchi obyektдан yaqinda bo'lsa kichik deb hisoblanadi. Bunday amalga oshirish varianti, berilgan ikkita nuqtadan qaysi biri kattaroq sferada

joylashganligini aniqlashga imkon beradi. Agar operatorlardan hech biri mantiqiy true qiymatini qaytarmasa, u holda ikkala nuqta bir xil sferada bo'ladi.

```
// < va > operatorlarini qayta yuklash
using System;
// Uch o'lchovli koordinatalarini saqlash uchun sinf
class ThreeD
{
    int x, y, z; // Uch o'lchovli koordinatalar
    public ThreeD() { x = y = z = 0; }
    public ThreeD(int i, int j, int k) { x = i; y = j; z = k; }
    // < operatorini qayta yuklash
    public static bool operator <(ThreeD op1, ThreeD op2)
    {
        if (Math.Sqrt(op1.x * op1.x + op1.y * op1.y + op1.z * op1.z) <
            Math.Sqrt(op2.x * op2.x + op2.y * op2.y + op2.z * op2.z))
            return true;
        else
            return false;
    }
    // > operatorini qayta yuklash
    public static bool operator >(ThreeD op1, ThreeD op2)
    {
        if (Math.Sqrt(op1.x * op1.x + op1.y * op1.y + op1.z * op1.z) >
            Math.Sqrt(op2.x * op2.x + op2.y * op2.y + op2.z * op2.z))
            return true;
        else
            return false;
    }
    // X, Y, Z koordinatalarini chiqarish.
    public void Show()
    {
        Console.WriteLine("(" + x + ", " + y + ", " + z + ")");
    }
}
class ThreeDDemo
{
    static void Main()
    {
        ThreeD a = new ThreeD(5, 6, 7);
        ThreeD b = new ThreeD(10, 10, 10);
        ThreeD c = new ThreeD(1, 2, 3);
        ThreeD d = new ThreeD(6, 7, 5);
        Console.WriteLine("a nuqta koordinatalari: ");
        a.Show();
        Console.WriteLine("b nuqta koordinatalari: ");
        b.Show();
        Console.WriteLine("c nuqta koordinatalari: ");
        c.Show();
        Console.WriteLine("d nuqta koordinatalari: ");
        d.Show();
        if (a > c) Console.WriteLine("a > c rost");
    }
}
```

```
if (a < c) Console.WriteLine("a < c rost");
if (a > b) Console.WriteLine("a > b rost");
if (a < b) Console.WriteLine("a < b rost");
if (a > d) Console.WriteLine("a > d rost");
else if (a < d) Console.WriteLine("a < d rost");
else
Console.WriteLine("a va d nuqtalar koordinatadan bir xil masofada joylashgan");
Console.ReadKey();
}
}
```

Dastur bajarilishi natijasida quyidagi natijani qaytaradi.

a nuqta koordinatalari: (5, 6, 7)

b nuqta koordinatalari: (10, 10, 10)

c nuqta koordinatalari: (1, 2, 3)

d nuqta koordinatalari: (6, 7, 5)

a > c rost

a < b rost

a va d nuqtalar koordinatadan bir xil masofada joylashgan